From a Scratch CNN to a Pretrained Transformer Encoder: An Iterative Study on Deep-Supervised Multi-Decoder Networks for Visual Saliency Prediction

Simone Freschet [2196384], Sebastiano Colombini [2197932]

***Abstract*—Visual saliency prediction, estimating where humans look in an image, is a long-standing computer vision problem with applications from image compression to autonomous driving and interface design. We document the design and iterative refinement of a deep-supervised, multi-decoder saliency network, trained from scratch on SALICON and evaluated cross-dataset on MIT1003. Starting from a literal replica of a VGG-based encoder-decoder trained with MSE, we progressively modernized the pipeline (Adam, batch normalization, a combined KL-divergence/correlation/similarity deep-supervision loss) and finally replaced the convolutional encoder with a pretrained Swin Transformer, yielding a hybrid model (v2.0) compared against its convolutional predecessor (v1.1). Preliminary MIT1003 results show comparable performance between the two, with the Transformer encoder not yet translating into the expected generalization gain, a result we analyze. Beyond the architecture, we report the iterative, trial-and-error process behind it (successes, dead ends and design decisions), which we consider as informative as the final numbers.**

***Index Terms*—Visual Saliency Prediction, Convolutional Neural Networks, Vision Transformers, Deep Supervision, Transfer Learning, SALICON, MIT1003.**

I. Introduction

Human visual attention is not distributed uniformly over a scene: observers consistently fixate on a small, predictable subset of regions. *Visual saliency prediction* estimates, for a given image, the distribution of human eye fixations as a continuous *saliency map*. Reliable saliency models support perceptually-driven image/video compression, thumbnailing, retargeting, advertising design, and attention priors for downstream vision and robotics pipelines.

This report:

(i) describes a multi-decoder, deep-supervised saliency network trained from scratch on SALICON with a combined KL-divergence/correlation/similarity loss applied to the fused output and three intermediate decoder branches (Section IV);

(ii) extends it by replacing the encoder with a pretrained Swin-Tiny Transformer while keeping decoder, fusion and loss fixed, isolating the effect of the encoder swap (Section V);

(iii) evaluates both cross-dataset on MIT1003, discussing where the expected Transformer gains did, and did not, materialize (Section VI, Section VII);

(iv) documents the iterative engineering process behind these models (Section III).

Structure. Section II positions our work in the literature. Section III describes the development process. Section IV and Section V detail the CNN and Transformer-encoder models, respectively. Section VI and Section VII describe the evaluation protocol and preliminary results, and Section VIII discusses findings and open work.

II. Related Work

CNN encoder-decoder saliency models. Our starting point is the VGG-based encoder-decoder of Wang and Shen [1], which we replicate as our first prototype (Section III-B); a closely related design is Ghariba et al.'s five-conv/five-deconv VGG-16 encoder-decoder [2]. Both rely on a pretrained encoder and a single reconstruction-style loss. Our from-scratch replica, trained with plain MSE, produced over-smoothed maps (Section III-B); this gap motivated our departure from a single pixel-wise loss toward a deep-supervised, multi-term saliency loss (Section IV-C).

Transfer learning from object recognition. Kümmerer et al. [3] show that VGG-19 features pretrained for object recognition, used *without fine-tuning*, already predict fixations competitively; direct evidence for the hypothesis in Section III-A. Their result is demonstrated only for convolutional backbones; in v2.0 (Section V) we test whether it extends to a pretrained Swin Transformer [4], substituted for our convolutional encoder while the rest of the pipeline is held fixed.

Transformer-based saliency. TranSalNet [5] augments a CNN backbone with bespoke, jointly-trained transformer fusion blocks, achieving state-of-the-art results on MIT300/SALICON at the cost of extra architectural and training complexity. v2.0 tests a cheaper alternative: an off-the-shelf pretrained Transformer *encoder* paired with our existing lightweight decoders, which better fits our hardware budget (Section III-E) and isolates the effect of the encoder alone.

Evaluation methodology. Yan et al. [6] and Bylinskii et al. [7] argue that no single saliency metric suffices, since location- and distribution-based metrics capture different

properties. We follow this by reporting three complementary distribution-based metrics (Section VI-B) and by testing cross-dataset on MIT1003 [8] rather than only on held-out SALICON data.

In short, we adopt the encoder-decoder topology and transfer-learning premise of [1]–[3], we replace the reconstruction loss with a deep-supervised saliency-specific loss and test the attention-based direction of [5] as a minimal, controlled encoder substitution rather than a bespoke fusion architecture.

III. Development Workflow

This section reconstructs *how* we arrived at the architectures of Section IV–Section V: several choices there (e.g. Adam over Nesterov momentum, or the three-term deep-supervision loss) are the direct consequence of an earlier iteration underdelivering, and are best read in that light. Tab. 1 summarizes the five iterations.

A. Literature review and initial hypotheses

The initial literature review (Section II) produced two working hypotheses that shaped every later decision: (i) encoders pretrained for object recognition should transfer well to saliency, even without fine-tuning [3]; (ii) attention-based models had already reached state-of-the-art saliency results, but through bespoke, jointly-trained transformer modules [5]; whether a plain pretrained Transformer *encoder*, dropped into a lightweight decoder design like ours, would bring comparable benefits was an open question, which we deferred to a later iteration (Section V) rather than testing immediately. We also read work on audio-visual saliency [9] as a possible future extension, not pursued here.

B. From a literal replica to a working baseline (Beta, v1.0)

Our first prototype ("Beta") literally replicated [1], trained from scratch with a pixel-wise MSE loss. Results were poor: over-smoothed maps with geometric artifacts. We attribute this mainly to training the VGG-based encoder entirely from scratch, unlike [1] itself, which initializes it from a pretrained network; this is an early, practical confirmation of hypothesis (i). This motivated **v1.0**: Adam replaced Nesterov momentum, and batch normalization was added, since the VGG backbone used in [1] predates batch normalization (VGG: 2014, batch normalization: 2015) and never included it. The MSE loss was also replaced with the deep-supervised KLD+CC+SIM loss of Section IV-C, applied to the fused output and every decoder branch.

C. Refining the baseline: v1.1

v1.1 deliberately left architecture and loss untouched and instead changed the input to the native, rectangular SALICON resolution (240×320) and added early stopping. We expected a clear improvement; gains were only marginal, suggesting the v1.0 loss/optimization changes already captured most of the achievable improvement at this model capacity.

D. A pretrained Transformer encoder: v2.0

v2.0 tests hypothesis (ii): the convolutional encoder is replaced with a pretrained Swin-Tiny Transformer [4], while decoders, fusion and loss are kept identical to v1.1 so any difference is attributable to the encoder (Section V). Our expectations were the highest of the project; after a long run, results were again only marginally different from v1.1 (Section VII), most plausibly due to the conservative differential learning rate and heavy backbone freezing (Section VIII).

E. Cross-dataset validation and environment

Since development only ever validated on held-out SALICON data, we built a MIT1003-based [8] test pipeline (Section VI), applied identically to both models, so the comparison in Section VII reflects generalization rather than in-distribution fit. All training ran on a single Colab T4 GPU (~15 GB VRAM); this budget directly motivated v2.0's backbone freezing and mixed precision.

TABLE 1: Timeline of the main development iterations.

Date	Stage	Key change
19 May	Setup	Repository and Colab/Drive pipeline.
20–29 May	Beta	Literature review; literal replica of [1], MSE loss, from scratch.
29 May–05 Jun	v1.0	Adam, batch norm, KLD+CC+SIM deep-supervision loss.
05–08 Jul	v1.1	Rectangular 240×320 input, early stopping.
10–13 Jul	v2.0	Encoder $\rightarrow$ pretrained Swin-Tiny; differential LR, freezing, AMP.
13–15 Jul	Testing	MIT1003 cross-dataset evaluation.

IV. Model v1.1: A Deep-Supervised Multi-Decoder CNN

Model v1.1 is the fully-convolutional model resulting from the iterations in Section III-B and Section III-C: an encoder producing features at three depths, three parallel decoders that independently upsample each feature map to a single-channel saliency estimate, and a final fusion module. Fig. 1 shows the pipeline, shared with v2.0 (Section V), which differs only in the encoder.

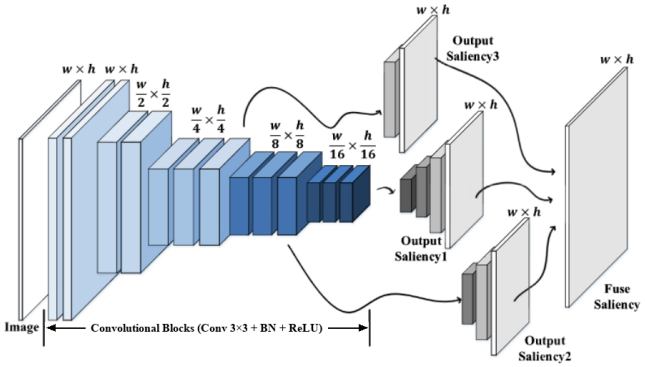


Fig. 1: v1.1 encoder-decoder pipeline (adapted from [1]).

A. Encoder

The encoder is a VGG-style CNN trained from scratch: five blocks of 3×3 convolutions (stride 1, padding 1), batch normalization and ReLU. The first four blocks end with 2×2 max-pooling, the fifth does not, so the last two taps share spatial resolution. Feature maps are extracted after blocks 3, 4 and 5 (Tab. 2) and passed to the three decoders below.

TABLE 2: Encoder configuration (v1.1). c: output channels, s: stride relative to the input.

Block	Layers	Output	Tap
1	Conv3-64, Conv3-64, MaxPool2	c=64, s=2	–
2	Conv3-128, Conv3-128, MaxPool2	c=128, s=4	–
3	(Conv3-256)×3, MaxPool2	c=256, s=8	F_3
4	(Conv3-512)×3, MaxPool2	c=512, s=16	F_4
5	(Conv3-512)×3	c=512, s=16	F_5

B. Decoders and fusion

Each feature map F_k ($k \in \{3,4,5\}$) is processed by a dedicated decoder D_k of identical structure: a 1×1 channel-reduction convolution, three blocks of 3×3 conv + BN + ReLU each interleaved with ×2 bilinear upsampling, and a final 1×1 conv + BN + ReLU collapsing to a single-channel map $S_k = D_k(F_k)$. The three maps are concatenated channel-wise and passed through `FinalCov`, a 1×1 (transposed) convolution followed by a sigmoid, giving the fused map

$$S = \sigma\big(\mathrm{Conv}_{1\times1}([S_3;\, S_4;\, S_5])\big) \in [0,1]^{H\times W}. \tag{1}$$

Combining three independently-trained estimates, rather than relying on the deepest feature map alone, is what motivates the deep-supervision loss below: each branch must produce a plausible estimate on its own.

C. Deep-supervision loss

Let P be a predicted map and G the ground-truth fixation density, both normalized to sum to one. We combine three complementary terms: the Kullback-Leibler divergence

$$\mathrm{KLD}(G,P) = \sum_i G_i \log\left(\epsilon + \frac{G_i}{\epsilon + P_i}\right), \tag{2}$$

the linear correlation coefficient

$$\mathrm{CC}(G,P) = \frac{\mathrm{cov}(G,P)}{\sigma_G\, \sigma_P}, \tag{3}$$

and the histogram similarity

$$\mathrm{SIM}(G,P) = \sum_i \min(G_i, P_i), \tag{4}$$

with ϵ a small constant avoiding division/log-of-zero in Eq. (2). These give a single per-map loss

$$\begin{aligned}\mathcal{L}(G,P) = w_{\mathrm{kld}}\,\mathrm{KLD}(G,P) + w_{\mathrm{cc}}\big(1-\mathrm{CC}(G,P)\big) \\ + w_{\mathrm{sim}}\big(1-\mathrm{SIM}(G,P)\big),\end{aligned} \tag{5}$$

with $w_{\mathrm{kld}} = w_{\mathrm{cc}} = w_{\mathrm{sim}} = 1$ throughout. The weights were introduced to leave room for rebalancing the three terms in later iterations, but the uniform setting proved effective from the start and was never revisited. Rather than applying Eq. (5) only to S, we apply *deep supervision*, summing it over the fused map and all three branches:

$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}(G,S) + \sum_{k\in\{3,4,5\}} \mathcal{L}(G,S_k). \tag{6}$$

This change was introduced in v1.0 (Section III-B) to replace an initial MSE loss; together with Adam and batch normalization, it was the single change most responsible for the qualitative jump from Beta to v1.0.

D. Dataset and training setup

The model is trained on SALICON [10] (10,000 train / 5,000 validation images), resized to 240 × 320, tensor-converted without normalization or augmentation; MIT1003 [8] is held out entirely for test time (Section VI).The 240 × 320 resolution was chosen because its 3:4 aspect ratio matches that of the SALICON images (480 × 640), so resizing preserves their proportions instead of deforming them as a square input would; it also better suits the rectangular, landscape-oriented format of most natural photographs.

TABLE 3: Training configuration, model v1.1.

Parameter	Value
Optimizer	Adam
Learning rate	1×10^{-3}
Batch size	16
Max. epochs	150
Early stopping	patience 15, $\Delta_{\min} = 10^{-4}$
Input resolution	240 × 320
Mixed precision	no

V. Model v2.0: Pretrained Transformer Encoder

Model v2.0 follows the same pipeline as v1.1 (Fig. 1), with the decoder (Section IV-B), fusion module (Eq. (1)), and loss function (Section IV-C) remaining unchanged. This section focuses on the differences introduced in v2.0, namely the encoder and the resulting training configuration.

A. Encoder

The scratch-trained VGG-style encoder of Section IV-A is replaced with `swin_tiny_patch4_window7_224`, a Swin Transformer [4] pretrained on ImageNet-1k, which merges shifted local-attention windows across four stages into a feature pyramid analogous to the v1.1 encoder's. As in the previous model, we extract the outputs of the last three stages, obtaining the feature maps F_3, F_4, and F_5, with 192, 384, and 768 channels at strides 8, 16, and 32, respectively (Tab. 4). Input images are resized to 224×224, matching the backbone's pretraining resolution.

TABLE 4: Encoder configuration (v2.0, Swin-Tiny). c: output channels, s: stride relative to the input.

Stage	Output	Tap
2	c=192, s=8	F_3
3	c=384, s=16	F_4
4	c=768, s=32	F_5

Because F_5 now has stride 32 rather than 16 (Tab. 2), Decoder_5 needs one extra $\times 2$ upsampling stage to match v1.1's output resolution; Decoder_3/Decoder_4 are unaffected. All other decoder, fusion and loss components are identical to Section IV.

B. Training setup and fine tuning

To keep the pretrained representation intact while allowing some task-specific adaptation, the backbone is frozen by default and only its last stage is fine-tuned. The head is trained at a learning rate ten times higher than the unfrozen backbone stage, following standard discriminative fine-tuning practice [11] for adapting pretrained features to a much smaller target dataset [12]. Since most of the backbone is frozen, memory pressure on the T4 GPU is dominated by head activations; we exploit this with a substantially larger batch size than v1.1, combined with automatic mixed precision (AMP) [13]. Tab. 5 summarizes the configuration; dataset, splits and evaluation protocol are identical to v1.1 (Section IV-D).

TABLE 5: Training configuration, model v2.0. Parameters not listed are identical to Tab. 3.

Parameter	Value
Optimizer	Adam (2 param. groups)
Head learning rate	3×10^{-4}
Backbone learning rate	3×10^{-5}
Backbone freezing	frozen, last stage unfrozen
Batch size	128
Input resolution	224×224
Mixed precision	yes (AMP)

VI. Test and Validation Protocol

A. Cross-dataset generalization test

Both models are trained and early-stopped exclusively on SALICON splits (Section IV-D). Since the SALICON test set ground-truth fixation maps are not publicly released, it is not possible to perform an objective evaluation on the official test split. Therefore, for a fair and reproducible comparison, both models are evaluated on the MIT1003 dataset [8], which contains 1,003 images with fixation data collected from 15 observers. MIT1003 provides an independent benchmark with a substantially different distribution of natural scenes than SALICON's context-driven web imagery, while remaining completely unseen during training and model selection. The same evaluation procedures are applied to both models on the entire MIT1003 dataset, ensuring that the comparison presented in Section VII isolates the effect of the encoder and the resizing in preprocessing rather than differences in the training or evaluation pipeline.

B. Evaluation metrics

Saliency evaluation is sensitive to metric choice, since different metrics emphasize different map properties and a model can score well on one and poorly on another [7]. We therefore report the same three complementary, distribution-based metrics used as loss terms in Section IV-C: KLD (Eq. (2), $\downarrow$), CC (Eq. (3), $\uparrow$) and SIM (Eq. (4), $\uparrow$), computed between the fused output S and the MIT1003 ground truth, plus the total loss $\mathcal{L}_{\text{total}}$ (Eq. (6)) used for checkpoint selection.

C. Model selection and early stopping

For both models, we evaluate the checkpoint corresponding to the lowest validation loss on the SALICON validation set. Training is performed for a maximum of 150 epochs, with early stopping triggered if the validation loss fails to improve by more than a predefined threshold for 15 consecutive epochs. To mitigate overfitting, the best-performing checkpoint (i.e., the last checkpoint showing an improvement in validation loss) is retained, while the subsequent epochs without improvement are discarded. As shown in Fig. 2 and Fig. 3, Model v1.1 converges in fewer epochs than Model v2.0. However, this difference should not be interpreted as an intrinsic property of the architectures alone, since the two models were trained with different learning rates, which also affects the convergence rate.

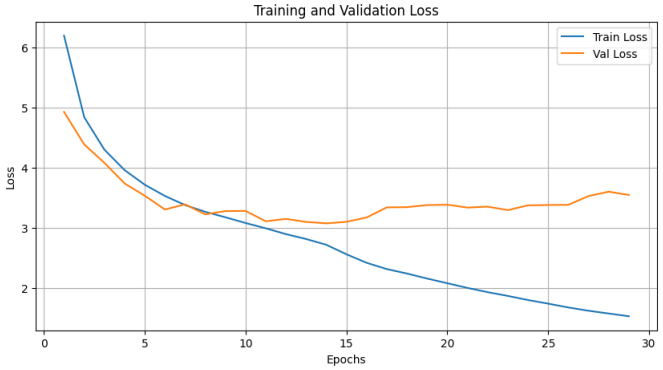


Fig. 2: Training loss vs Validation loss of Model v1.1

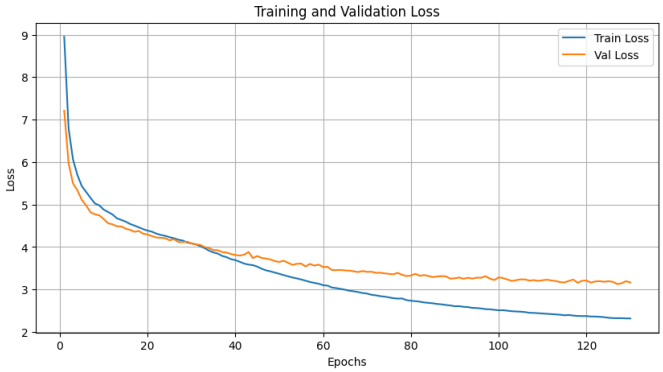


Fig. 3: Training loss vs Validation loss of Model v2.0

VII. Testing

A. Objective testing

Tab. 6 reports the metrics of Section VI-B on MIT1003 for the best checkpoint of each model.

TABLE 6: MIT1003 cross-dataset test results (best checkpoint, preliminary). ↓: lower is better, ↑: higher is better.

Model	$\mathcal{L}_{\text{total}}\downarrow$	KLD↓	CC↑	SIM↑
v1.1 (CNN encoder)	7.6305	**0.9229**	**0.6044**	**0.4809**
v2.0 (Swin-Tiny encoder)	**7.5088**	0.9436	0.5929	0.4609

At this stage, the two models exhibit very similar performance. While Model v2.0 achieves a marginally lower total loss, Model v1.1 performs slightly better on all three individual evaluation metrics. Given the small differences, these results do not support a clear ranking between the two architectures.

B. Subjective testing

As with many multimedia-related applications, subjective evaluation remains the gold standard for assessing perceptual quality. To complement the quantitative metrics, we conducted an informal user study by showing images from the test set to a group of friends and asking them to compare the predicted saliency maps with the corresponding ground truth. In several cases, participants reported preferring the model's prediction over the ground truth itself, highlighting the inherent subjectivity of saliency maps, which are derived from human fixation data.

These observations should not be considered conclusive, as the study did not follow the standardized protocols typically adopted for subjective quality assessment. In particular, no participant screening or training was performed, and responses were collected as qualitative comments rather than numerical ratings, preventing any meaningful statistical analysis. Fig. 4 and Fig. 5 present representative examples of the saliency maps produced by the proposed models.

C. Discussion

From a qualitative perspective, Model v2.0 requires more epochs to converge, and the mandatory resizing of the input images to a square resolution introduces a slight stretching effect in the predicted saliency maps. Nevertheless, its predictions tend to be more spatially defined and sharper than those of Model v1.1. Moreover, the current implementation employs the Tiny variant of the Swin Transformer, primarily due to computational constraints. Larger Swin variants have significantly greater representational capacity and could potentially provide a measurable improvement in performance under the same training framework.

Overall, despite the architectural differences between a scratch-trained CNN encoder and a pretrained Transformer encoder, the two models achieve broadly comparable results under the current experimental setup.

VIII. Concluding Remarks

A. Summary

We presented the iterative development of a deep-supervised, multi-decoder network for visual saliency prediction, progressing from a scratch-trained baseline to improved optimization, training strategies, and a pretrained Swin-Tiny encoder. The final CNN- and Transformer-based variants were further evaluated in a cross-dataset setting on MIT1003 to assess their generalization capability.

B. Interpretation

The largest improvement observed throughout the project came from the changes introduced in v1.0, namely the adoption of a more suitable optimization strategy and the combined KLD/CC/SIM deep-supervision loss, rather than from increasing the model capacity. The subsequent higher-resolution training in v1.1 and the replacement of the CNN encoder with a pretrained Swin-Tiny Transformer in v2.0 resulted in only marginal performance differences, despite the Transformer-based model representing the most substantial architectural change.

Two non-exclusive factors may explain the limited impact of the Transformer encoder. First, the conservative differential learning rate and near-total backbone freezing (Section V-B) may not allow enough task-specific adaptation. Second, the current implementation uses the Tiny variant of the Swin Transformer, selected to satisfy the available computational resources. Although computationally efficient, this model has substantially lower representational capacity than the Small, Base, or Large variants, which may be better suited to capture the complex semantics required for visual saliency prediction.

Overall, this constitutes a useful finding: simply replacing a convolutional encoder with a higher-capacity pretrained architecture is not sufficient to guarantee better performance. Instead, the loss formulation, supervision strategy, training configuration and the choice of backbone size all play a crucial role in determining the final performance.

C. Limitations and future work

Several directions could be explored to further improve the proposed models. First, the transfer-learning strategy adopted for Model v2.0 could be revisited by progressively unfreezing additional Transformer stages or performing a more extensive search over the differential learning rates applied to the backbone and decoder. Evaluating larger Swin Transformer variants (e.g., Swin-S or Swin-B) would also help determine whether the limited improvement observed with Swin-Tiny is primarily due to its reduced capacity.

From a training perspective, introducing standard data augmentation techniques and input normalization may further improve robustness and cross-dataset generalization. The evaluation protocol could also be extended by adding location-based metrics (AUC, NSS) alongside KLD/CC/SIM used in this work.

Finally, some architectural improvements remain unexplored, like experimenting with alternative feature fusion strategies and using attention-based decoder modules.

D. What we learned

In the end, we are really pleased with the final results. The moment that satisfied us the most was the jump from Beta

to v1.0, which brought the biggest improvement of the whole project in both output quality and performance.

The most frustrating part was dealing with Google Colab: the limited runtime (especially painful for v2.0) forced us to build a snapshot mechanism to keep track of the model across disconnections and account changes.

Still, we were happy to learn to juggle so many different architectures, from classical CNNs all the way to Transformers. Above all, v1.0 was where we added our own touch to something that already existed, combining what we learned in class with the research we did on our own; making it better was fun and rewarding at the same time.

E. Declaration on Generative AI

During the preparation of this work, the authors utilized AI-based tools exclusively to assist in the drafting, refining, and formatting of the manuscript's text, ensuring that all core concepts and scientific contributions remain original. Following the generation or editing of any content, the authors reviewed, curated, and edited the material, and take full responsibility for the accuracy and integrity of the final paper.

REFERENCES

[1] W. Wang and J. Shen, "Deep Visual Attention Prediction," *IEEE Transactions on Image Processing*, vol. 27, pp. 2368–2378, May 2018.

[2] B. Ghariba, M. S. Shehata, and P. McGuire, "Visual Saliency Prediction Based on Deep Learning," *Information*, vol. 10, no. 8, p. 257, 2019.

[3] M. Kümmerer, T. S. A. Wallis, L. A. Gatys, and M. Bethge, "DeepGaze II: Reading Fixations from Deep Features Trained on Object Recognition," *arXiv preprint arXiv:1610.01563*, 2016.

[4] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[5] J. Lou, H. Lin, D. Marshall, D. Saupe, and H. Liu, "TranSalNet: Towards Perceptually Relevant Visual Saliency Prediction," *Neurocomputing*, vol. 494, pp. 455–467, 2022.

[6] F. Yan, C. Chen, and P. Xiao, "Review of Visual Saliency Prediction: Development Process from Neurobiological Basis to Deep Models," *Applied Sciences*, vol. 12, no. 1, p. 309, 2022.

[7] Z. Bylinskii, T. Judd, A. Oliva, A. Torralba, and F. Durand, "What Do Different Evaluation Metrics Tell Us About Saliency Models?," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 3, pp. 740–757, 2019.

[8] T. Judd, K. Ehinger, F. Durand, and A. Torralba, "Learning to Predict Where Humans Look," in *IEEE International Conference on Computer Vision (ICCV)*, (Kyoto, Japan), Sept. 2009.

[9] Y. Zhu, H. Duan, K. Zhang, Y. Zhu, X. Zhu, L. Teng, X. Min, and G. Zhai, "How Does Audio Influence Visual Attention in Omnidirectional Videos? Database and Model," *arXiv preprint arXiv:2408.05411*, 2024.

[10] M. Jiang, S. Huang, J. Duan, and Q. Zhao, "SALICON: Saliency in Context," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1072–1080, 2015.

[11] J. Howard and S. Ruder, "Universal Language Model Fine-tuning for Text Classification," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, (Melbourne, Australia), July 2018.

[12] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in *Advances in Neural Information Processing Systems 27 (NIPS)*, (Montreal, Canada), Dec. 2014.

[13] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," 2018.

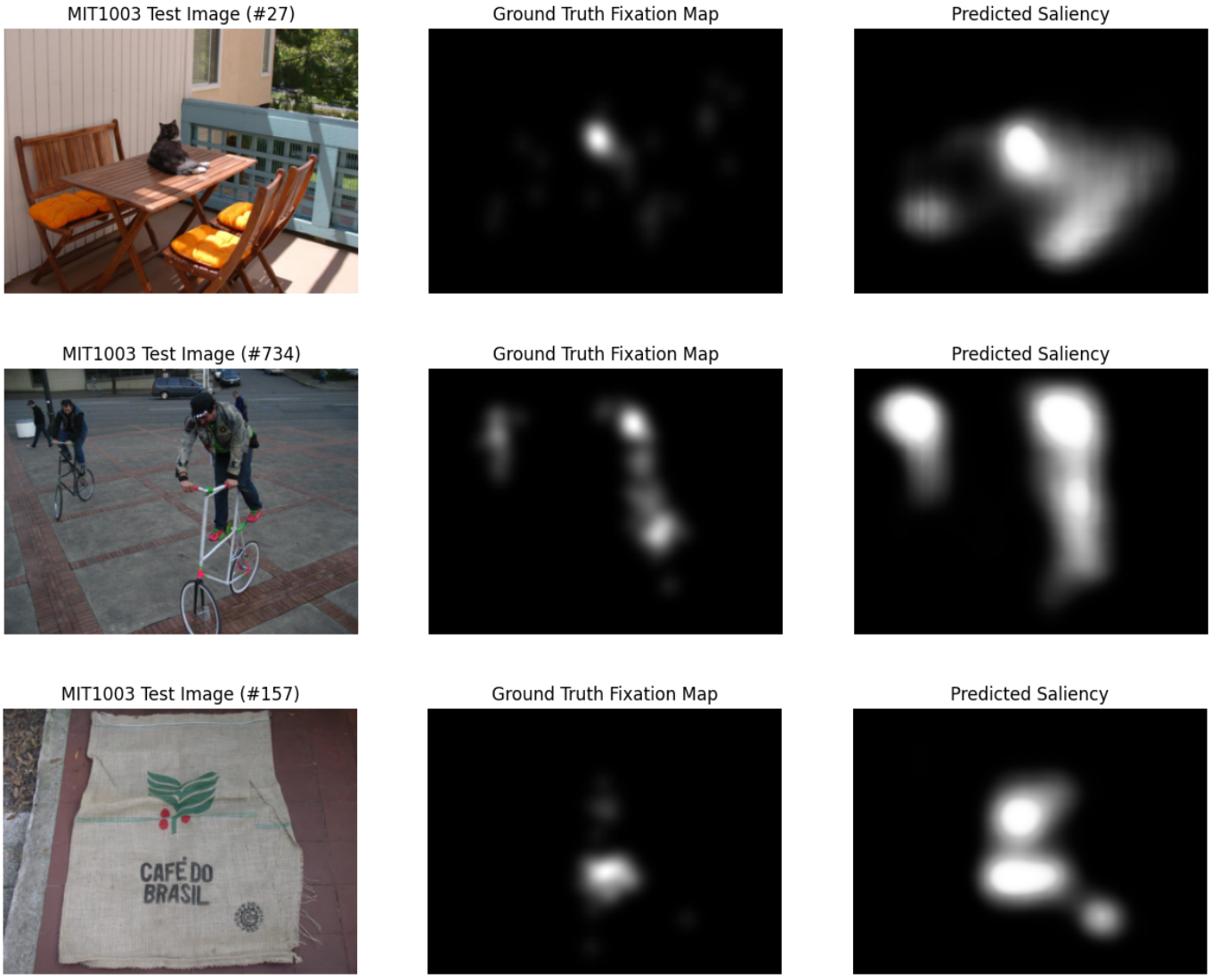


Fig. 4: Original / Ground Truth / Predicted from Model v1.1

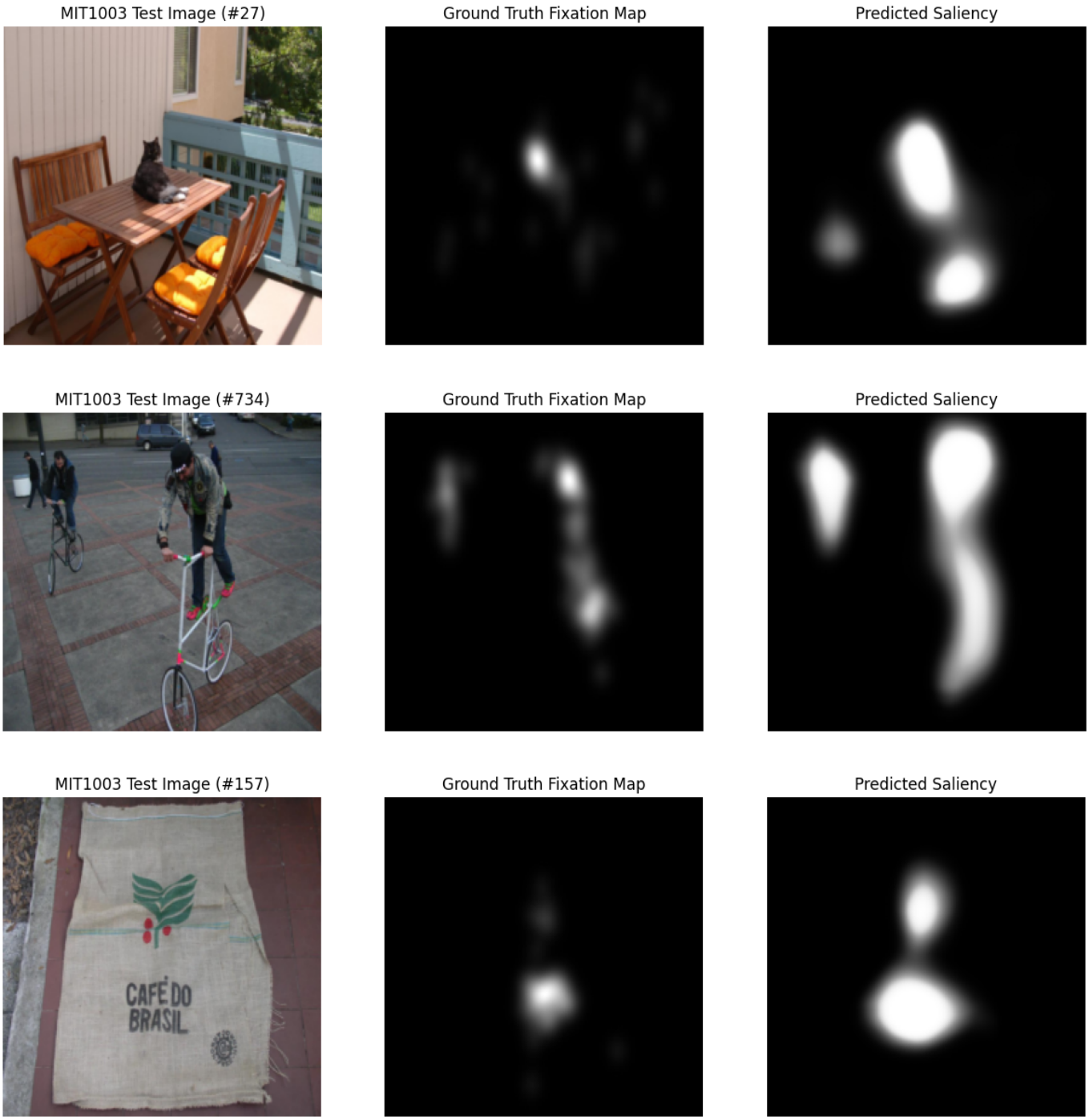


Fig. 5: Original / Ground Truth / Predicted from Model v2.0